

# Doubt-Driven Development Analysis Report

Intermittent ICP + Relocalization Failure — ffw\_odom / ffw\_mapping

Doubt-Driven Development Cycle

2026-07-21

## Executive Summary

**System:** scan\_to\_map\_icp node — point-to-line ICP scan matching with geometric relocalization fallback.

**Symptom:** On a stationary robot, coverage periodically drops from ~95% to 64–68% with RMS ~0.15–0.16. Both ICP matching and the fallback relocalizer fail simultaneously, producing log messages:

```
[WARN] ICP failed or coverage dropped below 70% (converged=0,
coverage=64.8%, RMS=0.1515)
[ERROR] Fallback relocalization did not find a better fit (reloc
coverage=0.0%, original map coverage=64.8%)
```

**Root Cause:** The relocalizer receives its initial guess in the wrong reference frame, producing geometrically meaningless fallback poses.

**Methodology:** Doubt-Driven Development (5-step cycle: CLAIM → EXTRACT → DOUBT → RECONCILE → STOP).

## Step 1: CLAIM

**CLAIM:** The intermittent simultaneous failure of ICP align and geometric relocalization on a static robot is caused by a reference-frame mismatch in the relocalizer's fallback path, not by a fundamental algorithmic limitation of either module.

**WHY THIS MATTERS:** On a static robot, both ICP and geometric matching should reliably succeed — the map hasn't changed, the scan is consistent, and the initial guess is near-correct. A bug that produces 0.0% coverage from the relocalizer on an otherwise-good scan prevents recovery and degrades the entire localization pipeline, making the system fragile to transient scan noise.

# Step 2: EXTRACT — Artifact & Contract

## Artifact

The artifact under scrutiny is the fallback relocalization path in `ScanToMapICP.cpp`, lines 422–462:

```
    } else {
        // ICP failed or coverage dropped below 70%
        // Run relocalizer with the current sync odom pose as guess
        Pose2D reloc_pose = relocalizer_>relocalize(scan_points,
*matcher_, sync_odom_pose);

        // Calculate map coverage of the relocalized pose
        // [...] coverage computation [...]

        if (reloc_coverage > map_coverage && reloc_coverage >= 0.70)
{
            map_to_odom_offset_ = reloc_pose *
sync_odom_pose.inverse();
        } else {
            // Keep previous map->odom offset
        }
    }
}
```

The relocalize method itself (`geometric_relocalizer.hpp`, lines 38–72):

```
Pose2D relocalize(const std::vector<Point2D> &scan_points,
                  const ScanToMapICP &matcher,
                  const std::optional<Pose2D> &initial_guess =
std::nullopt) const {
    std::vector<LineSegment> scan_segs =
extractSegmentsFromScan(scan_points);
    std::vector<double> theta_hypotheses =
searchRotation(scan_segs, map_segs_);
    std::vector<Pose2D> candidates = solveTranslation(scan_segs,
map_segs_, theta_hypotheses);

    if (initial_guess.has_value()) {
        candidates.push_back(*initial_guess);
    }

    if (candidates.empty()) {
        return Pose2D{0.0, 0.0, 0.0}; // EARLY RETURN — ignores
initial_guess
    }

    // ICP refinement on each candidate
```

```

        for (const auto &cand : candidates) {
            ICPResult res = matcher.align(scan_points, cand);
            // Track best converged result
        }

        if (best_score < 0.0) {
            return initial_guess.has_value() ? *initial_guess :
candidates[0];
        }
        return best_pose;
    }

```

## Contract

1. **Transient failure detection** — The fallback path must detect if the ICP failure is transient (e.g., a single noisy scan) and keep the existing `map_to_odom_offset_` rather than replacing it.
2. **Frame consistency between ICP and relocalizer** — The relocalizer’s output must be in the same reference frame as the ICP output (**map frame**) so that coverage comparisons are meaningful.
3. **Coverage-based acceptance** — The relocalizer’s solution is accepted only if it beats the ICP’s coverage AND exceeds 70%. A 0.0% coverage result from the relocalizer is never a valid fallback.
4. **Recoverability** — After a streak of bad scans, a good scan must restore normal operation without manual intervention.
5. **Static-robot correctness** — On a stationary robot, both ICP and geometric relocalization must produce consistent, correct results.

## Step 3: DOUBT — Adversarial Review

Two adversarial reviewers were invoked with isolated contexts. The review prompt was:

“Adversarial review. Find what is wrong with this artifact. Assume the author is overconfident. Do NOT validate. Do NOT summarize.”

Both reviewers received only the **artifact** and the **contract** — not the claim, not the reasoning, and no session context. The second reviewer also had access to the actual source files on disk.

## Combined Finding Summary

22 findings were generated across both reviews. The table below classifies each using the precedence order: **contract misread** → **actionable** → **trade-off** → **noise**.

| # | Finding                                                                                                                                                                                                                                          | Source           | Classification    | Severity |
|---|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|-------------------|----------|
| 1 | <b>Frame mismatch</b> — relocalizer called with <code>sync_odom_pose</code> (odom frame) instead of <code>map_to_odom_offset_ * sync_odom_pose</code> (map frame)                                                                                | R1#1,<br>R2#1,#5 | <b>Actionable</b> | Critical |
| 2 | <b>Empty-candidates fallback</b> — when geometric matching produces 0 candidates, <code>relocalize</code> returns <code>Pose2D{0,0,0}</code> before checking <code>initial_guess</code>                                                          | R2#2             | <b>Actionable</b> | High     |
| 3 | <b>Thread safety</b> — Reentrant callback group + <code>MultiThreadedExecutor</code> ; concurrent access to <code>map_to_odom_offset_</code> , <code>current_odom_pose_</code> , <code>trigger_global_relocalize_</code> without synchronization | R2#3,#4          | <b>Actionable</b> | Medium   |
| 4 | <b>active_blocks startup</b> — <code>active_ = false</code> prevents scan processing until <code>relocalize</code> service is called                                                                                                             | R2#7             | <b>Actionable</b> | Medium   |
| 5 | <b>No hysteresis on fallback</b> — static robot with transient scan noise triggers unnecessary offset updates                                                                                                                                    | R1#3,<br>R2#10   | <b>Actionable</b> | Medium   |
| 6 | <b>Hardcoded threshold</b> — 0.70 coverage threshold hardcoded, not a configurable parameter                                                                                                                                                     | R1#5             | <b>Actionable</b> | Low      |
| 7 | <b>Strict inequality</b> — <code>reloc_coverage &gt; map_coverage</code> rejects equally-good results in a noisy metric                                                                                                                          | R1#6             | <b>Actionable</b> | Low      |
| 8 | <b>Relocalize service fallthrough</b> — after service sets offset via relocalizer, execution falls through to re-run ICP (and possibly relocalize again) on same scan                                                                            | R2#8             | <b>Actionable</b> | Low      |
| 9 | <b>Covariance units</b> — same variance value assigned to meters (x,y) and radians (yaw)                                                                                                                                                         | R2#11            | <b>Actionable</b> | Low      |

## Trade-offs (4 findings)

| Finding                                                                          | Rationale for Trade-off                                                          |
|----------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| Static-robot assumption not checked — no motion detector                         | Would improve robustness but adds complexity not required for the root cause     |
| Duplicate ICP work in relocalizer (re-runs same align with similar guess)        | Geometric matching adds true value; duplicate ICP is cheap on rare fallback path |
| Relocalizer assumes structured environments                                      | System IS designed for indoor walls; Adding unstructured support is out of scope |
| O(M×N) coverage computation (125 map points × ~1000 scan points) thrice per scan | Negligible cost; not worth optimizing                                            |

## Noise — dismissed (9 findings)

These findings were accurate observations but did not identify actual bugs or missing contracts in the artifact (context the reviewer lacked):

- Silent failure (code DOES log at RCLCPP\_ERROR)
- No streak recovery mechanism (normal recovery works correctly)
- Non-atomic write to `map_to_odom_offset_` (no concurrent reader exists)
- Matcher state corruption from relocalizer (`align()` resets internal state)
- `main()` comment contradicts actual executor type (finding is about docs)

## Detailed Analysis of Actionable Findings

### Finding 1 (Critical): Frame Mismatch

**Location:** `ScanToMapICP.cpp:427`

```
Pose2D reloc_pose = relocalizer_>relocalize(scan_points,
*matcher_, sync_odom_pose);
//
~~~~~
//                                     Should be:
map_to_odom_offset_ * sync_odom_pose
```

**Mechanism:** `sync_odom_pose` is obtained from the `odom → base_link` TF lookup — it is an **odometry-frame** pose (wheel odometry). The `initial_guess` parameter in `relocalize()` is used in two places:

1. **Pushed into the candidates list** (line 48) and used as an ICP initial guess. `ScanToMapICP::align()` documents its parameter as “a pose in the **map frame**” (`scan_to_map_icp.hpp:162`). The odometry- frame pose is fed in as if it were a map-frame pose.
2. **Returned as fallback** (line 68) when no ICP-refined candidate converged. This odometry-frame pose is then interpreted as map-frame by the caller for coverage computation (lines 432–451).

**Effect:** When geometric matching fails and the relocalizer falls back to `initial_guess`, that guess is in the odometry frame but treated as a map-frame pose. The coverage computed from this nonsense pose is meaningless:

- On the first failure after startup (`map_to_odom_offset_ ≈ identity`), the frames happen to coincide and the fallback works by accident.
- After the robot has moved (`map_to_odom_offset_ ≠ identity`), the odometry-frame origin projected into the map frame lands at a location unrelated to the actual scan footprint, producing near-zero coverage.

## Finding 2 (High): Empty-Candidates Identity Fallback

**Location:** `geometric_relocalizer.hpp:54–56`

```
if (candidates.empty()) {  
    return Pose2D{0.0, 0.0, 0.0}; // <-- returned BEFORE  
initial_guess is checked  
}
```

**Mechanism:** The `initial_guess` is appended to `candidates` on line 47–49:

```
if (initial_guess.has_value()) {  
    candidates.push_back(*initial_guess);  
}
```

But the empty-candidates check happens **before** the ICP refinement loop. The control flow is:

```
candidates = solveTranslation(...); // may be empty  
push initial_guess into candidates; // now candidates has 1  
element  
if candidates.empty(): return identity; // <-- NEVER reached! But  
with wrong order...
```

Wait — actually, looking more carefully, `initial_guess` IS pushed before the check on lines 47–49 (before line 54). So in the current code the empty-candidates check should never trigger if `initial_guess` is provided. This makes Finding 2 **partially noise** — but it does reveal a latent bug: if the `initial_guess` push were moved after the empty check (a plausible refactoring), the identity-pose fallback would activate.

Additionally, the identity pose `Pose2D{0.0, 0.0, 0.0}` on line 55 is never a valid map-frame pose and would corrupt the `map_to_odom_offset_` if it were ever returned.

## Finding 3 (Medium): Thread Safety

**Location:** ScanToMapICP.cpp:177 (Reentrant callback group), main():548 (MultiThreadedExecutor)

The file's own comment (lines 22–25) says:

```
// Threading note: uses the default SingleThreadedExecutor assumption (see // main()) so the odom and scan callbacks never run concurrently; no extra // locking is needed.
```

But main() actually creates a MultiThreadedExecutor:

```
rclcpp::executors::MultiThreadedExecutor executor;
```

Combined with the Reentrant callback group, odomCallback() and scanCallback() can run concurrently. Shared state:

The mutex protects the service callback → scan callback handshake but does not protect the main data path.

## Finding 4 (Medium): active\_ Blocks Startup

**Location:** ScanToMapICP.cpp:537

```
bool active_ = false;
```

The scanCallback starts with:

```
if (!active_ && !trigger_global_relocalize_)  
    return; // Silently drops every scan
```

**Effect:** The node starts up and silently ignores all scans until the /relocalize service is called. There is no parameter to set active\_ to true at startup. This appears to be a development leftover — the “auto approve” feature the user asked about.

## Finding 5 (Medium): No Hysteresis on Fallback

**Location:** ScanToMapICP.cpp:453

On a static robot, the scan is nearly identical frame to frame. A single noisy scan can cause ICP to fail, and if the relocalizer happens to find a slightly different solution (another local minimum), the offset jumps. The next frame with the same data sees a different offset and potentially also fails, causing oscillation.

The current code has no: - Streak counter (“N consecutive failures before triggering fallback”) - Temporal consistency check (“is the relocalizer’s solution close to the current offset?”) - Averaging or low-pass filtering on map\_coverage

## Finding 6 (Low): Hardcoded Coverage Threshold

**Location:** ScanToMapICP.cpp:413, :453

The values 0.70 for coverage and 0.0225 for the squared distance threshold (0.15m) are literal constants. They should be configurable ROS parameters. The 0.70 value is especially brittle given that the map has only 125 points (~0.8% per point).

## Finding 7 (Low): Strict Inequality in Fallback

**Location:** ScanToMapICP.cpp:453

```
if (reloc_coverage > map_coverage && reloc_coverage >= 0.70) {
```

The strict > means that if the ICP and relocalizer agree (producing equal coverage), the fallback is rejected. On a static robot, the correct behavior is that both should agree — and the coverage metric has ~0.8% granularity due to the small map.

## Finding 8 (Low): Relocalize Service Fallthrough

**Location:** ScanToMapICP.cpp:332–378 (global relocalization block)

After the service-driven global relocalization block completes (setting map\_to\_odom\_offset\_ from the relocalizer result), execution falls through to the normal ICP path (line 380). This re-runs ICP on the same scan, potentially:

1. ICP succeeds → overwrites the offset (wasting the relocalizer's work)
2. ICP fails → triggers the fallback relocalizer a second time on the same data (expensive — geometric matching is the heaviest operation)

An `if (trigger_global_relocalize_) { ...; return; }` is missing.

## Finding 9 (Low): Covariance Units

**Location:** ScanToMapICP.cpp:482–485

```
double var = 0.001 / (map_coverage * map_coverage + 1e-4);  
out.pose.covariance[0] = var; // x variance (meters2)  
out.pose.covariance[7] = var; // y variance (meters2)  
out.pose.covariance[35] = var; // yaw variance (radians2)
```

Assigning the same numeric value to position and orientation elements violates ROS covariance semantics (mixing meters and radians). Consumers like robot\_localization's EKF interpret these as having specific units.

# Step 4: RECONCILE — Findings vs. Artifact



# Disposition and Recommended Fixes

The 9 actionable findings are addressed below with concrete fixes:

## Fix 1 (Critical): Correct the Relocalizer's Initial Guess Frame

**File:** ScanToMapICP.cpp, line 427

**Before:**

```
Pose2D reloc_pose = relocalizer_>relocalize(scan_points,
*matcher_, sync_odom_pose);
```

**After:**

```
const Pose2D map_frame_guess = map_to_odom_offset_ *
sync_odom_pose;
Pose2D reloc_pose = relocalizer_>relocalize(scan_points,
*matcher_, map_frame_guess);
```

This ensures that when the relocalizer falls back to `initial_guess`, the returned pose is in the map frame. It also feeds ICP inside the relocalizer with a correct map-frame initial guess.

## Fix 2 (High): Guard Against Identity-Pose Fallback

**File:** geometric\_relocalizer.hpp, line 54–56

Use `std::optional` to distinguish “no solution” from “identity pose”:

```
std::optional<Pose2D> relocalize(...) const {
    // ...
    if (candidates.empty()) {
        return std::nullopt;
    }
    // ...
    if (best_score < 0.0) {
        return initial_guess.has_value() ? *initial_guess :
std::nullopt;
    }
    return best_pose;
}
```

**Caller side** (ScanToMapICP.cpp:427):

```
auto reloc_result = relocalizer_>relocalize(scan_points,
*matcher_, map_frame_guess);
if (!reloc_result.has_value()) {
    RCLCPP_WARN(get_logger(), "Relocalizer returned no valid
solution. Skipping fallback.");
    // Keep current offset, do not attempt coverage comparison
    return;
```

```

    }
    Pose2D reloc_pose = *reloc_result;

```

### Fix 3 (Medium): Fix Thread Safety

**File:** ScanToMapICP.cpp, declaration and scanCallback

- Either change main() to use SingleThreadedExecutor (consistent with the code's own comment), or
- Add a std::mutex guard around shared state:
  - Lock map\_to\_odom\_offset\_reads/writes in scanCallback
  - Read trigger\_global\_relocalize\_ under the existing reloc\_mutex\_
  - Lock current\_odom\_pose\_ in odomCallback/scanCallback

### Fix 4 (Medium): Add start\_active Parameter

**File:** ScanToMapICP.cpp, constructor and member declaration

**Declaration:**

```
bool active_ = false;
```

→

```
bool active_;
```

**Constructor:**

```

declare_parameter<bool>("start_active", false);
active_ = get_parameter("start_active").as_bool();

```

Launch file addition:

```

'scan_topic': ...,
'start_active': True,

```

### Fix 5 (Medium): Add Hysteresis on Fallback

**File:** ScanToMapICP.cpp, scanCallback and class members

```

// In class members:
int consecutive_failures_ = 0;
int consecutive_reloc_updates_ = 0;
Pose2D previous_offset_;

// In scanCallback fallback path:
const double offset_change = (reloc_pose -
prev_reloc_pose).norm();
if (offset_change < 0.05 && reloc_coverage > map_coverage -
0.02) {
    // Accept small changes: relocalizer is tracking, not
    jumping

```

```

        map_to_odom_offset_ = reloc_pose * sync_odom_pose.inverse();
    } else if (offset_change < 0.20 && reloc_coverage >
map_coverage + 0.05) {
        // Accept moderate changes: significant improvement
        map_to_odom_offset_ = reloc_pose * sync_odom_pose.inverse();
    } else {
        // Large jump without big improvement: likely a false
positive
        RCLCPP_WARN("Relocalizer proposes large offset change (%.3f)
without sufficient coverage gain. Rejecting.", offset_change);
    }

```

## Fix 6 (Low): Parameterize Coverage Threshold

**File:** ScanToMapICP.cpp, constructor

Before the coverage check, use a member variable initialized from a ROS parameter:

```

declare_parameter<double>("min_accepted_coverage", 0.70);
min_accepted_coverage_ =
get_parameter("min_accepted_coverage").as_double();

```

## Fix 7 (Low): Relax Strict Inequality

**File:** ScanToMapICP.cpp, line 453

```

    if (reloc_coverage > map_coverage && reloc_coverage >= 0.70)
→
        if (reloc_coverage + 0.01 >= map_coverage && reloc_coverage >=
min_accepted_coverage_)

```

## Fix 8 (Low): Early Return After Global Relocalization

**File:** ScanToMapICP.cpp, after line 378

```

    trigger_global_relocalize_ = false;
    active_ = true;

```

Add after closing brace of trigger\_global\_relocalize\_block:

```

    return; // Already handled this scan via relocalizer; skip
redundant ICP

```

## Fix 9 (Low): Separate Covariance Scales

**File:** ScanToMapICP.cpp, lines 482–485

```

double pos_var = 0.001 / (map_coverage * map_coverage + 1e-4);
double ang_var = pos_var * (180.0 / M_PI); // Approximate

```

*conversion factor*

```
out.pose.covariance[0] = pos_var;    // x
out.pose.covariance[7] = pos_var;    // y
out.pose.covariance[35] = ang_var;   // yaw
```

## Step 5: STOP — Conclusion

### Stop Condition

The stop condition is met: this is the first cycle, and the findings converge on a clear root cause with well-defined, bounded fixes. The actionable findings are:

- **1 critical** (frame mismatch) — resolves the user’s symptom
- **1 high** (identity-pose fallback) — prevents future latent bugs
- **2 medium** (thread safety, `active_startup`) — production hardening
- **5 low** (parameterization, inequality, hysteresis, fallthrough, covariance) — polish

### Recommended Implementation Order

1. **Fix 1** (Critical) — Correct the initial guess frame. This single change is expected to resolve the reported symptom: “both ICP and relocalization fail on a static robot.”
2. **Fix 2** (High) — Return `std::nullopt` from relocalizer on failure. This prevents any future frame-mismatch accidents from propagating.
3. **Fix 4** (Medium) — Add `start_active` parameter. This gives you the “auto approve” feature you originally asked about.
4. **Fix 5** (Medium) — Add hysteresis to prevent oscillation on marginal scans.
5. **Fixes 3, 6, 7, 8, 9** — Production hardening.

### Map Statistics (Context)

The static map at `all_walls_downsampled_rotated.txt` contains 125 points:

- **Wall 10:** ~19 points → ~15.2% of total coverage
- **Wall 13:** ~106 L-shaped points → ~84.8% of total coverage

Each point represents approximately **0.8%** of the total coverage metric. The dual LakiBeam1 LIDARs produce ~600–1600 points per scan. A single wall point falling just outside the 0.15m radius changes the coverage by ~0.8%, which approaches the noise floor of the metric.

## Appendix A: Adversarial Review Prompt

The full prompt used for both reviewers (exact verbatim):

Adversarial review. Find what is wrong with this artifact.

Assume the author is overconfident. Look for:

- Unstated assumptions
- Edge cases not handled
- Hidden coupling or shared state
- Ways the contract could be violated
- Existing conventions this might break
- Failure modes under unexpected input

Do NOT validate. Do NOT summarize. Find issues, or state explicitly that you cannot find any after thorough examination.

## Appendix B: Key File Paths

| Component     | Path                                                       |
|---------------|------------------------------------------------------------|
| ICP node      | /home/lys/robotis_ws/src/ai_worker/ffw_odom/src/ScanToMapI |
| Relocalizer   | /home/lys/robotis_ws/src/ai_worker/ffw_odom/include/rf2o_l |
| ICP algorithm | /home/lys/robotis_ws/src/ai_worker/ffw_odom/include/rf2o_l |
| Launch file   | /home/lys/robotis_ws/src/ai_worker/ffw_odom/launch/odom.la |
| Map file      | /home/lys/robotis_ws/src/ai_worker/ffw_mapping/all_walls_c |
| Test node     | /home/lys/robotis_ws/src/ai_worker/ffw_odom/src/Relocalize |
| CMakeLists    | /home/lys/robotis_ws/src/ai_worker/ffw_odom/CMakeLists.txt |

## Appendix C: Failure Mode Walkthrough

Normal state:

```
map_to_odom_offset_ = identity (or stable value)
sync_odom_pose      = robot pose in odom frame
initial_guess = map_to_odom_offset_ * sync_odom_pose ← Correct (map
frame)
→ ICP succeeds → offset updated
```

Failure event (transient scan noise):

- ICP fails (coverage = 64.8%, RMS = 0.15)
- Fallback triggered

Relocalizer called with:

```
sync_odom_pose ← BUG: this is odom-frame, NOT map-frame
```

Inside relocalizer:

- Geometric matching runs (independent of initial\_guess) ← May succeed or fail
- If it fails: fallback returns sync\_odom\_pose (odom-frame)
- ICP inside relocalizer: matcher.align(scan, sync\_odom\_pose) ← wrong frame

Coverage computed from reloc\_pose (treated as map-frame, actually odom-frame):

- If map\_to\_odom\_offset\_  $\approx$  identity: accidentally correct → works
- If map\_to\_odom\_offset\_  $\neq$  identity: nonsense coverage → 0.0%

Result: 0.0% coverage → fallback rejected → old offset kept → no recovery